程序报告

学号：　　姓名：

一、问题重述

（简单描述对问题的理解，从问题中抓住主干，必填）

==========

这个问题是要使用 DQN 算法让机器人自动探索迷宫，DQN 使用深度神经网络来学习机器人在不同状态下的动作价值，从而选择最佳动作到达目标点。这个问题的主干为使用经验重放记忆（如使用 ReplayDateset 类存储实验数据），实现 DQN 算法。

二、设计思想

（所采用的方法，有无对方法加以改进，该方法有哪些优化方向（参数调整，框架调整，或者指出方法的局限性和常见问题），伪代码，理论结果验证等… **思考题，非必填**）

==========

三、代码内容

（能体现解题思路的主要代码，有多个文件或模块可用多个"====="隔开，必填）

==========

```
class Robot(QRobot):
    valid_action = ['u', 'r', 'd', 'l']

    ''' QLearning parameters'''
    '''明白 epsilon 和 gamma 的意义，调节参数观察效果'''
    epsilon = 0.5  # 初始贪心算法探索概率
    gamma = 0.94  # 公式中的 γ

    EveryUpdate = 1  # the interval of target model's updating

    """some parameters of neural network"""
    '''调节网络参数'''
    target_model = None
    eval_model = None
    learning_rate = 1e-3 # 参数更新的学习率
    step = 1  # 记录训练的步数

    def __init__(self, maze):
        """
        初始化 Robot 类
        :param maze:迷宫对象
```

```
        """
        super(Robot, self).__init__(maze)
        '''设置奖励值'''
        maze.set_reward(reward={
            "hit_wall": 10.,
            "destination": -50.,
            "default": 1.,
        })
        self.maze = maze
        self.maze_size = maze.maze_size

        """build network"""
        self.target_model = None
        self.eval_model = None
        self._build_network()

        """create the memory to store data"""
        max_size = max(self.maze_size ** 2 * 3, 1e4)
        self.memory = ReplayDataSet(max_size=max_size)

    def _build_network(self):
        seed = 0
        random.seed(seed)

        """build target model"""
        self.target_model = Model(state_size=2, action_size=4, seed=seed)

        """build eval model"""
        self.eval_model = Model(state_size=2, action_size=4, seed=seed)

        """build the optimizer"""
        self.optimizer = paddle.optimizer.Adam(learning_rate=self.learning_rate,parameters=self.eval_model.parameters())

    def target_replace_op(self):
        """
            Soft update the target model parameters.
            θ_target = τ*θ_local + (1 - τ)*θ_target
        """
        """ replace the whole parameters"""
        self.target_model.set_state_dict(self.eval_model.state_dict())

    def _choose_action(self, state):
```

```
        state = np.array(state, dtype='float32')
        state = paddle.to_tensor(state)
        if random.random() < self.epsilon:
            action = random.choice(self.valid_action)
        else:
            self.eval_model.eval()
            with paddle.no_grad():
                q_next = self.eval_model(state).numpy()    # use target model choose action
            self.eval_model.train()

            action = self.valid_action[np.argmin(q_next).item()]
        return action

    def _learn(self, batch: int = 1):
        if len(self.memory) < batch:
            print("the memory data is not enough")
            return
        state, action_index, reward, next_state, is_terminal = self.memory.random_sample(batch)

        """ convert the data to tensor type"""

        state = paddle.to_tensor(state,dtype='float32')
        action_index = paddle.to_tensor(action_index,dtype='int32')
        reward = paddle.to_tensor(reward,dtype='float32')
        next_state = paddle.to_tensor(next_state,dtype='float32')
        is_terminal = paddle.to_tensor(is_terminal,dtype='int32')

        self.eval_model.train()
        self.target_model.eval()

        """Get max predicted Q values (for next states) from target model"""
        Q_targets_next = self.target_model(next_state).detach().min(1).unsqueeze(1)

        """Compute Q targets for current states"""

        Q_targets = reward + self.gamma * Q_targets_next * (1 - is_terminal)


        """Get expected Q values from local model"""
        self.optimizer.clear_grad()

        Q_expected = self.eval_model(state)
        action_index_onehot = fluid.layers.one_hot(action_index, 4)
        action_index_onehot = fluid.layers.cast(action_index_onehot, dtype='float32')
```

```
        Q_expected =
fluid.layers.reduce_sum(fluid.layers.elementwise_mul(action_index_onehot, Q_expected), dim=1)

        """Compute loss"""
        loss = F.mse_loss(Q_expected, Q_targets)
        loss_item = loss.item()
        """ Minimize the loss"""
        loss.backward()
        self.optimizer.step()

        """copy the weights of eval_model to the target_model"""
        self.target_replace_op()

        return loss_item

    def train_update(self):
        state = self.sense_state()
        action = self._choose_action(state)
        reward = self.maze.move_robot(action)
        next_state = self.sense_state()
        is_terminal = 1 if next_state == self.maze.destination or next_state == state else 0

        self.memory.add(state, self.valid_action.index(action), reward, next_state, is_terminal)

        """--间隔一段时间更新 target network 权重--"""
        if self.step % self.EveryUpdate == 0:
            self._learn(batch=1)

        """---update the step and epsilon---"""
        self.epsilon = max(0.2, self.epsilon * 0.995)

        self.step += 1
        return action, reward

    def test_update(self):
        state = np.array(self.sense_state(), dtype=np.int16)
        state = paddle.to_tensor(state,dtype='float32')

        self.eval_model.eval()
        with paddle.no_grad():
            q_value = self.eval_model(state).cpu().numpy()

        action = self.valid_action[np.argmin(q_value).item()]
        reward = self.maze.move_robot(action)
```

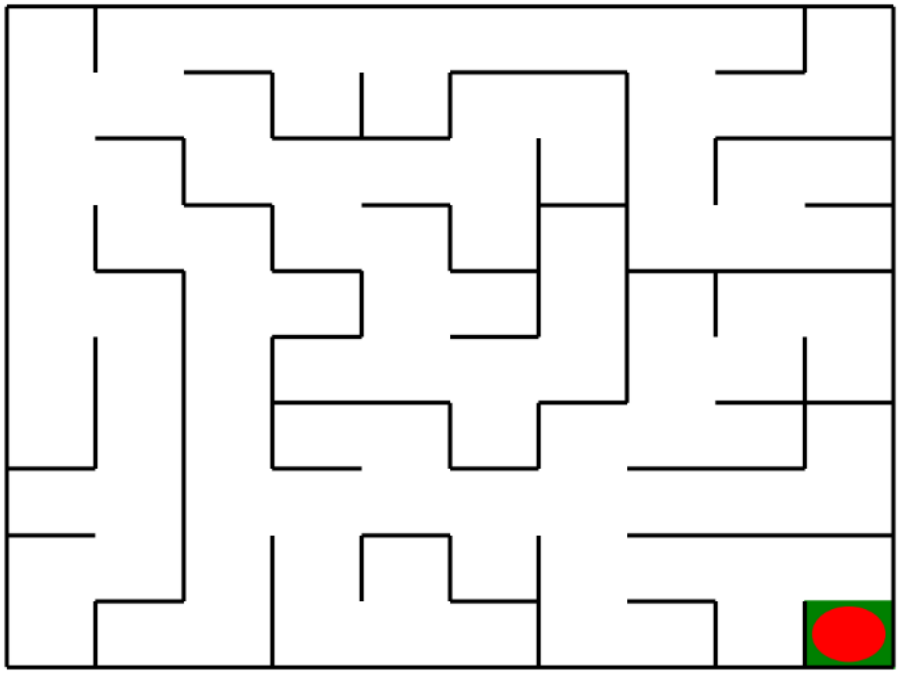
```
        return action, reward
```

四、实验结果

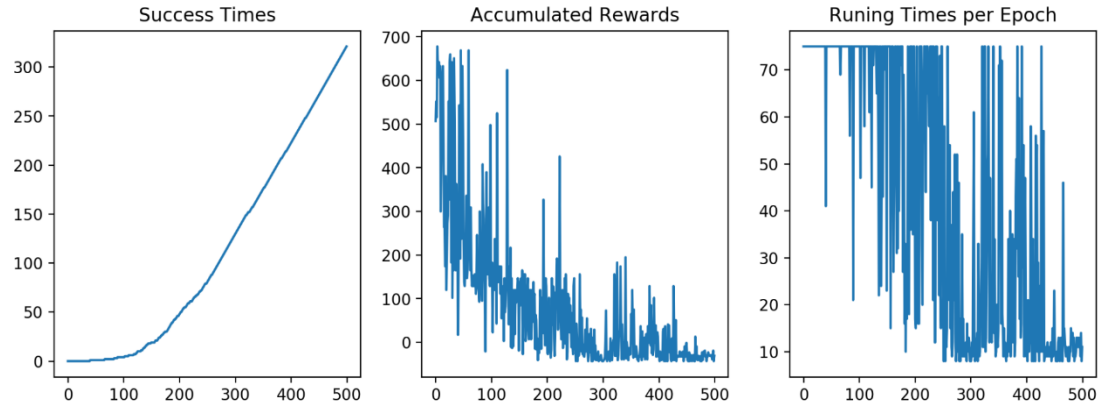
（实验结果，必填）

==========

```
搜索出的路径: ['d', 'd', 'r', 'd', 'r', 'd', 'd', 'd', 'd', 'r', 'r', 'r', 'r', 'd', 'r', 'r', 'r', 'd']
恭喜你，到达了目标点
```



```
<Figure size 640x480 with 1 Axes>
Maze of size (10, 10)
```



五、总结

（自评分析（是否达到目标预期，可能改进的方向，实现过程中遇到的困难，从哪些方面可以提升性能，模型的超参数和框架搜索是否合理等），**思考题，非必填**）

==========